

Course: Data Structures (CSE CS203A)

Assignment V: Tree

Due date: 2025.12.30 23:59:59

**Important Notice – Use of AI Tools**

In this assignment, you must use at least one AI assistant (e.g. ChatGPT, Gemini, Claude, Grok, M365 Copilot) as a learning tool to help you:

- review definitions,
- compare tree variants, and
- organize your report.

You are not allowed to let the AI directly produce your final diagrams or final report content without your own understanding and rewriting.

You must log all AI prompts and services used (see “AI Usage Log” section below).

**1. Goal of This Assignment**

In the lectures, we introduced the concept of the tree as a data structure, starting from the general tree and then moving to more specialized forms.

In this assignment, you will:

- Understand and clearly define:
  - General tree
  - Binary tree
  - Complete binary tree
  - Binary search tree (BST)
  - AVL tree
  - Red-Black tree
  - Max heap
  - Min heap
- Build a hierarchy and transformation path from the general tree to these variants, and explain how each variant adds more structure or constraints.
- Use a fixed list of integers to construct multiple tree variants and visualize them.
- Choose one real-world application for each tree type and explain why that data structure fits the application.
- Practice using AI tools as study companions and keep a simple Q&A log.

**2. Given Data**

Use the following 20 integers as the input data for all your tree constructions:

37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160

You will reuse this same sequence for every tree type (binary tree, complete binary tree, BST, AVL, Red-Black, max heap, min heap).

### 3. Deliverables

Please complete your work in the Student Worksheet Companion and upload it to the YZU Portal System.

Your report should include the following parts:

a. Definitions (Concept Review)

Provide clear, concise definitions for each of the following:

1. General tree
2. Binary tree
3. Complete binary tree
4. Binary search tree (BST)
5. AVL tree
6. Red-Black tree
7. Max heap
8. Min heap

You are encouraged to use AI tools to help you understand these concepts, but you must rewrite the definitions in your own words.

b. Hierarchy and Transformation of Tree Variants

Based on the definitions above, build a “tree family hierarchy” that shows how these structures are related. For example:

- General tree → Binary tree
- Binary tree → Complete binary tree / Binary search tree
- BST → AVL tree / Red-Black tree
- Binary tree → Max heap / Min heap

Tasks:

- Draw a diagram or flow chart that shows the transformation or specialization path: general tree → binary tree → complete binary tree → BST → AVL/Red-Black, etc.
- For each arrow (transformation), briefly explain:
  - What new constraint or property is added?
  - e.g., “Binary tree = tree with at most 2 children per node”,  
“BST = binary tree with left < root < right”,  
“AVL = BST with strict height-balance rule”, etc.

c. Tree Construction with the Given Integers

Using the given 20 integers, construct the following tree variants:

1. Binary tree
2. Complete binary tree
3. Binary search tree (BST)

4. AVL tree
5. Red-Black tree
6. Max heap
7. Min heap

Important Hint / Restriction:

- For these trees, you must use tree visualization tools (e.g., online visualizers or software) to build and display the tree.
- You may not ask AI tools to directly generate the final tree pictures for you.
- Instead:
  - Use AI only to help you understand algorithms,
  - Then apply those algorithms in a visualizer (or your own implementation).

What to submit for this part:

For each tree type:

- A snapshot (image) of the constructed tree.
- The URL / name of the visualization tool you used.
- A short note on how you inserted the integers (e.g., “insert in the given order as BST”, “build max heap using heapify”, etc.).

d. Application Example for Each Tree

For each of the following:

1. Binary tree
2. Complete binary tree
3. Binary search tree (BST)
4. AVL tree
5. Red-Black tree
6. Max heap
7. Min heap

Choose one application (real-world or system-level) and explain:

1. Application description
  - e.g., priority scheduling, dictionary lookup, memory allocation, database indexing, etc.
2. Why this tree structure fits
  - What property of this data structure makes it suitable?
  - Example:
    - Max heap → good for priority queue because the largest element is always at the root, so extracting max is efficient.
    - Red-Black tree → good for standard library maps/sets because it guarantees  $O(\log n)$  operations even under many insertions/deletions.

Your explanation should show that you understand the link between the data structure and its use case.

e. Report Layout and Organization

You are free to design the layout of your report, but it should:

- Be well-structured (use sections, headings, tables, and diagrams).
- Have a clear flow from:
  - definitions →
  - hierarchy/transformation →
  - constructed trees →
  - applications →
  - AI usage log.
- Be easy for another student to read and learn from.

Feel free to use AI to suggest a good outline, but you must decide and finalize the layout yourself.

f. AI Usage Log (Q&A Table)

Every time you use an AI copilot service for this assignment, record:

- Index (1, 2, 3, ...)
- Prompt (what you asked)
- Service (e.g., ChatGPT, Gemini, Copilot, ...)

Example log table:

| Index | Prompt                                                                                                                                                                      | Service |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| 1     | Assist me to have the definition of general tree, binary tree, complete binary tree, binary search tree, AVL tree, red-black tree, max heap and min heap for self-learning. | ChatGPT |
| 2     | Explain the difference between AVL tree and Red-Black tree in terms of balancing strategy and use cases.                                                                    | Gemini  |
| ...   | ...                                                                                                                                                                         | ...     |

Place this table at the end of your report.

4. Evaluation (100 pts)

A possible breakdown (you can adjust if needed):

- a. Concept definitions (20 pts)
  - Correctness and clarity of all 8 tree type definitions.
- b. Hierarchy & transformation explanation (20 pts)
  - Clear diagram / explanation of how each tree variant evolves from the general tree.
  - Correct identification of constraints/invariants.
- c. Tree constructions & visualizations (25 pts)
  - Correct constructions for each tree type using the given integers.
  - Proper screenshots and tool URLs.

- Consistent insertion / heap-building strategy descriptions.
- d. Applications & explanations (20 pts)
  - One application per tree type.
  - Clear explanation linking data structure properties to the application.
- e. Report organization & AI usage log (15 pts)
  - Logical report structure and readability.
  - AI log completeness (all prompts listed with service names).
  - Thoughtful use of AI as a learning assistant, not as a copy-paste generator.

Course: Data Structures (CSE CS203A)

Assignment V: Tree

Student Worksheet Companion

Due date: 2025.12.30 23:59:59

## Academic Integrity and AI Usage Statement

In this assignment, you must use AI tools (such as ChatGPT, Gemini, Claude, Grok, M365 Copilot, etc.) as learning assistants, but you must also take full responsibility for understanding and organizing your own work.

### 1. Permitted Use of AI Tools

You may use AI to:

- Review or clarify definitions and concepts.
- Compare different tree data structures.
- Get suggestions for report layout or examples.
- Ask for explanations of algorithms (e.g., BST insertion, AVL rotation, heapify process).

You should read, think about, and rewrite the content in your own words.

### 2. Not Permitted

- Do not copy/paste AI-generated content directly as your final answer.
- Do not ask AI to draw the final diagrams or directly produce the final tree screenshots.
- Do not ask AI to complete the whole assignment report for you.

### 3. Your Responsibility

- You are responsible for understanding the definitions and algorithms.
- You are responsible for verifying whether AI answers are correct or not.
- You must produce your own original explanations and diagrams.

### 4. AI Usage Log

- You must record all AI queries related to this assignment.
- At the end of your report, include an AI Usage Log table with: Index, Prompt, AI service name.

By submitting this assignment, you acknowledge that you have used AI tools only as study aids, and that the final content of this assignment represents your own understanding and work.

## Section 1. Definitions of Tree Variants

Task: Write your own definitions for each tree type. You may use AI for learning, but rewrite in your own words.

### 1. General Tree

Definition: 子節點的數量不像 binary tree 一樣有限制，可多個子節點。

## 2. Binary Tree

Definition: 每個節點最多只有 2 個子節點(左子樹和右子樹)。

## 3. Complete Binary Tree

Definition: 除了最後一層以外，每一層的節點都已經填滿，填入的順序是由左到右，不能跳著填。

## 4. Binary Search Tree (BST)

Definition: 可以是空樹，並且任意節點的左子樹所有節點 < 該節點 < 右子樹所有節點。

## 5. AVL Tree

Definition: 一種高度平衡的 binary search tree，樹中任意節點的左子樹和右子樹高度差的絕對值都不能超過 1

## 6. Red-Black Tree

Definition: 靠標籤節點的方式來平衡樹，紅節點的子節點必須是黑的

## 7. Max Heap

Definition: 每個父節點大於等於子節點，最大值在根

## 8. Min Heap

Definition: 每個父節點小於等於子節點，最小值在根

## Section 2. Tree Family Hierarchy and Transformations

Task: Show how these structures are related (general  $\rightarrow$  specialized). Use a simple diagram and explanations of what constraints are added at each step.

## 2.1 Tree Family Diagram

You may draw this by hand and paste a photo, or use drawing tools.

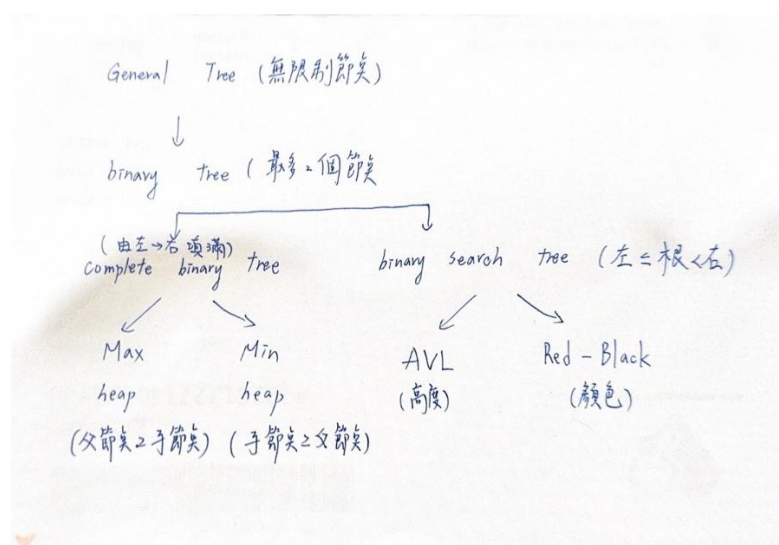
Suggested chain example (you may extend or adjust):

General Tree  $\rightarrow$  Binary Tree  $\rightarrow$  Complete Binary Tree

Binary Tree  $\rightarrow$  Binary Search Tree  $\rightarrow$  AVL / Red-Black

Binary Tree  $\rightarrow$  Max Heap / Min Heap

Your Diagram:



## 2.2 Explanation of Transformations

Fill in what new property or constraint is added at each step.

| From         | To                   | New property / constraint added |
|--------------|----------------------|---------------------------------|
| General Tree | Binary Tree          | 最多 2 個節點                        |
| Binary Tree  | Complete Binary Tree | 填滿，順序左到右                        |
| Binary Tree  | Binary Search Tree   | 左<根<右                           |
| BST          | AVL Tree             | 高度平衡，高度差 $\leq 1$               |
| BST          | Red-Black Tree       | 紅節點子節點為黑                        |
| Binary Tree  | Max Heap             | 父>子                             |
| Binary Tree  | Min Heap             | 父<子                             |

## Section 3. Tree Constructions Using Given Integers

Given integers (fixed for all parts):

37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160

Task: For each tree type below, construct the tree using these integers, take a screenshot of the tree from your chosen tool, record the tool name/URL, and describe the insertion / heap-building procedure.

### 3.1 Binary Tree

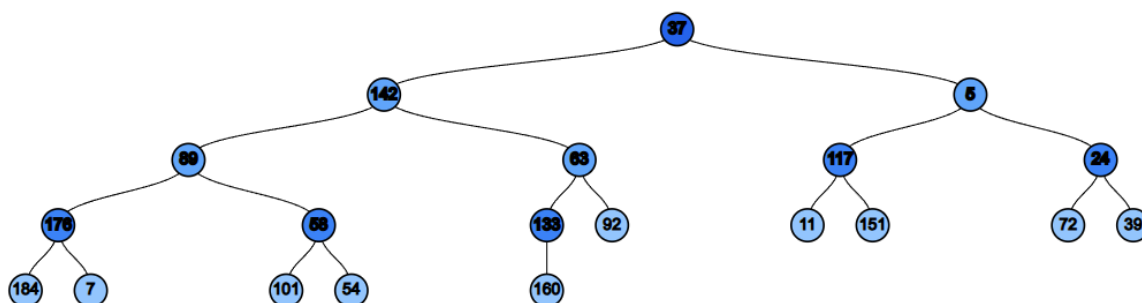
Tool name / URL:

[Binary tree visualizer and converter](#)

Construction / insertion description:

依照給定整數的順序，從上到下、由左到右插入節點

Screenshot of Binary Tree (paste below):





### 3.2 Complete Binary Tree

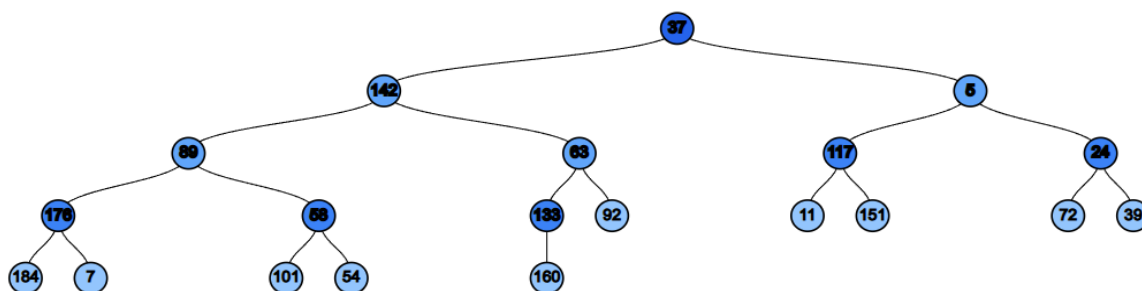
Tool name / URL:

[Binary tree visualizer and converter](#)

Construction / insertion description:

依照給定順序逐層由左至右插入節點，確保除最後一層外每一層都是滿的，最後一層由左至右填滿。

Screenshot of Complete Binary Tree (paste below):



### 3.3 Binary Search Tree (BST)

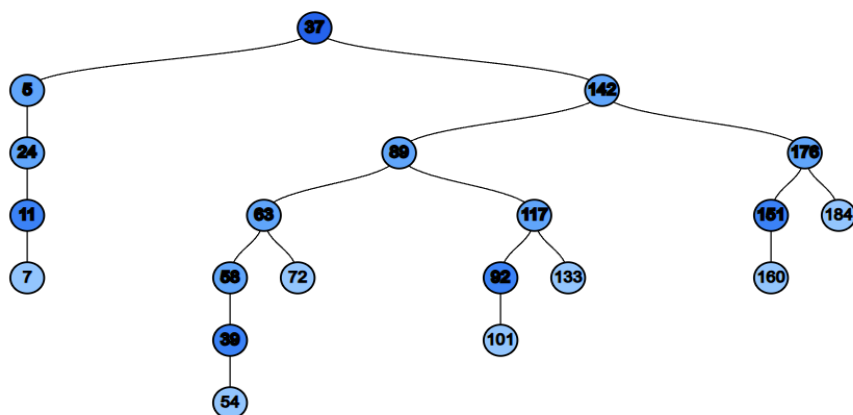
Tool name / URL:

[Binary tree visualizer and converter](#)

Insertion rule (e.g., "insert in given order using BST rules"):

按照給定順序插入整數，並遵守 BST 規則：小於目前節點的值放在左子樹，大於目前節點的值放在右子樹

Screenshot of BST (paste below):



### 3.4 AVL Tree

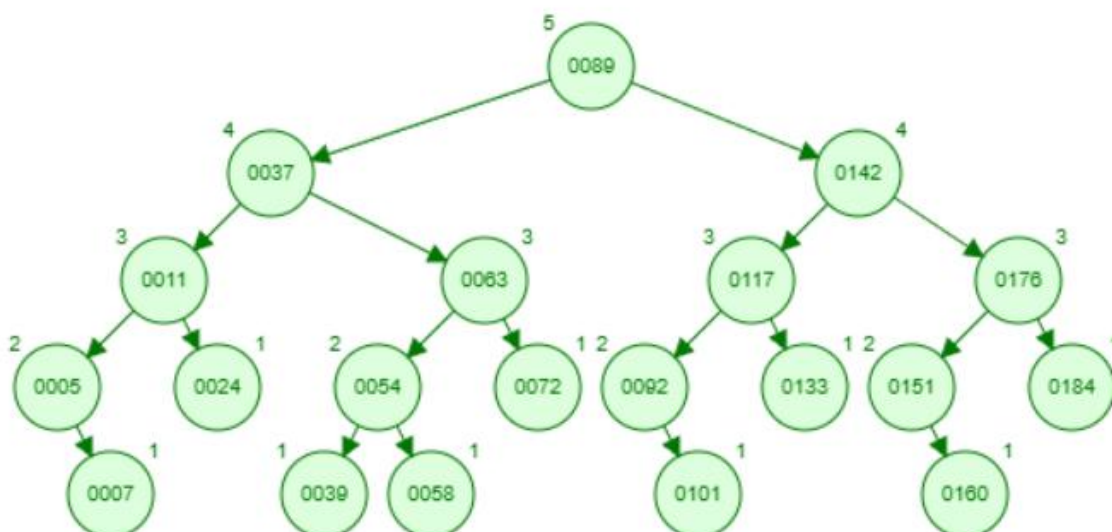
Tool name / URL:

[Usf ca data structure visualizations](https://www.usfca.edu/data-structure-visualizations/)

Insertion & balancing description:

按照 BST 規則插入整數，每次插入後檢查節點的高度差。

Screenshot of AVL Tree (paste below):



### 3.5 Red-Black Tree

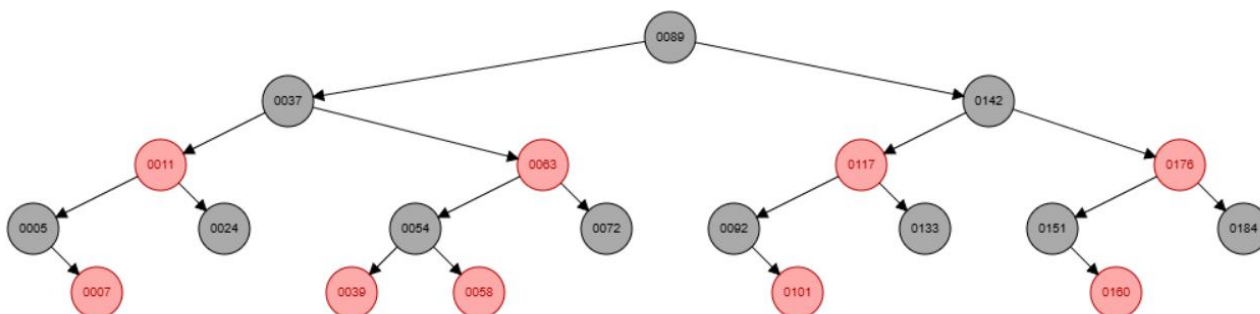
Tool name / URL:

[usf ca data structure visualizations](https://www.usfca.edu/data-structure-visualizations/)

Insertion & balancing description:

按照 BST 規則插入新節點，並將新節點標記為紅色。若出現違反紅黑樹性質的情況，則透過重新著色與旋轉操作來恢復樹的平衡。

Screenshot of Red-Black Tree (paste below):



### 3.6 Max Heap

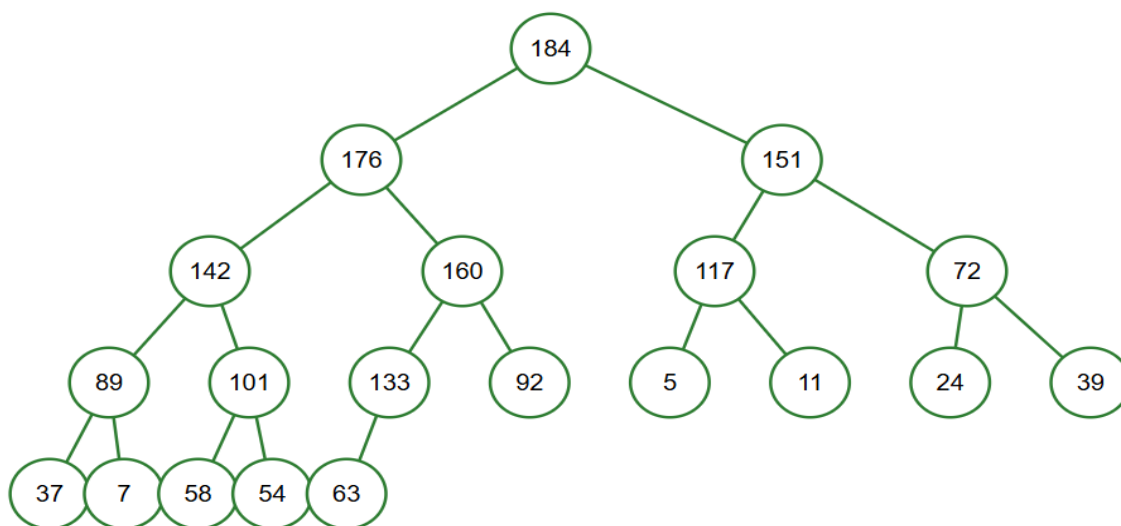
Tool name / URL:

[Max heap simulator](#)

Construction / heap-building description (e.g. heapify, insert-and-sift-up):

使用完全二元樹結構插入節點，並在每次插入後進行向上調整，確保每個父節點的值都大於或等於其子節點。

Screenshot of Max Heap (paste below):



### 3.7 Min Heap

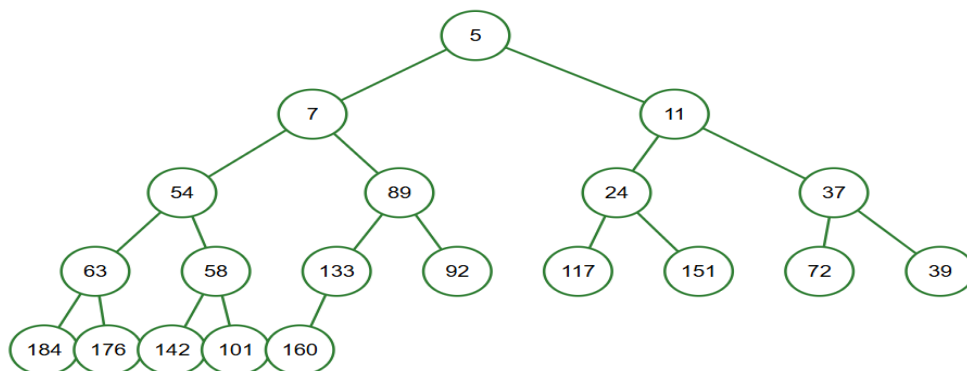
Tool name / URL:

[Min heap simulator](#)

Construction / heap-building description:

使用完全二元樹結構插入節點，並在每次插入後進行向上調整，確保每個父節點的值都小於或等於其子節點。

Screenshot of Min Heap (paste below):



### Section 4. Application Examples

Task: For each tree type, choose one application and explain why this tree is suitable.

| Tree Type            | Application Example (name / context) | Why this tree fits (properties that matter)       |
|----------------------|--------------------------------------|---------------------------------------------------|
| Binary Tree          | Expression Tree                      | 二元樹每個節點最多有兩個子節點，剛好對應運算子與兩個運算元，適合用來表示與計算數學運算式。     |
| Complete Binary Tree | Array-based representation           | 完全二元樹結構緊密，可以直接用陣列儲存節點，節省空間並方便存取父子節點。              |
| Binary Search Tree   | Dictionary Lookup                    | BST 具有左小右大的排序特性，能有效進行搜尋、插入與刪除操作。                  |
| AVL Tree             | Database Indexing                    | AVL 樹保持嚴格的高度平衡，使搜尋操作維持在 $O(\log n)$ ，適合需要快速查詢的情境。 |
| Red-Black Tree       | Std Map / Set                        | 紅黑樹在插入與刪除時平衡成本較低，能保證 $O(\log n)$ 效能，適合動態資料結構。     |
| Max Heap             | Priority Queue                       | 最大堆積中最大值永遠在根節點，可快速取得最高優先權的元素。                     |
| Min Heap             | Task Scheduling                      | 最小堆積中最小值位於根節點，適合快速取得最早或最小成本的任務。                   |

### Section 5. Reflection on Tree Family and Performance (Optional but recommended)

Among BST, AVL, and Red-Black trees, which one would you pick for:

Mostly search (few updates)? Why?

如果操作大多是搜尋，幾乎沒有插入或刪除會用 AVL 樹，因為它維持嚴格的高度平衡，能確保搜尋效率穩定在  $O(\log n)$ 。

Frequent insertions and deletions? Why?

如果需要頻繁進行插入與刪除操作會用紅黑樹，因為它在維持平衡時所需的旋轉次數較少，適合動態資料的情境。

If you must store these 20 integers for static search only (no updates), which structure or representation would you prefer (sorted array + binary search, BST, AVL, etc.)? Why?

sorted array+ binary search，實作簡單、空間使用效率高，且搜尋速度快。

### Section 6. AI Usage Log (Required)

Task: Record every time you ask an AI assistant about this assignment.

| Index | Date / Time | AI Service (ChatGPT, Gemini, etc.) | Your Full Prompt / Question |
|-------|-------------|------------------------------------|-----------------------------|
|-------|-------------|------------------------------------|-----------------------------|

Student ID: 1131535

Student Name: 蔡承諺

|   |             |         |                                       |
|---|-------------|---------|---------------------------------------|
| 1 | 12/29 22:09 | CHATGPT | 逐行翻譯成中文然後告訴我作業<br>要做甚麼                |
| 2 | 12/30 00:15 | GEMINI  | 紅黑數是甚麼                                |
| 3 | 12/30 00:43 | gemini  | 找一下 可以用的 visualization tool           |
| 4 | 12/30 02:18 | gemini  | BST, AVL, and Red-Black trees 的差<br>別 |

You may extend this table as needed.